have to touch (and you must not) the **next* and **prev* pointers. The kernel handles this. Thus, from a user point of view, the *timer_list* provides an expiration duration, a callback function and user-provided context. The *expires* field represents the *jiffies* value when the timer is expected to run. The user can initialise the timer by simply calling *setup_timer*, which sets up the callback and user-provided context. Also, the user can set the values of *data* and *function* in the timer and call *init_timer* (which is called internally by *setup_timer*). The prototypes are given below:

```
void init_timer( struct timer_list *timer );
void setup_timer( struct timer_list *timer, void (*function)
(unsigned long), unsigned long data );
```

The next step is to set the expiration time. This can be done by calling *mod_timer*. Timers are automatically deleted upon expiration, but to leave it hanging after the module has been unloaded may cause the kernel to hang. This also means that you can add the timer again from the handler itself.

The *cleanup_module* function calls *timer_pending* to detect whether the timer has expired or not. It returns true if the timer is pending. If you want to extend the timeout to acquire the following behaviour, issue the following commands:

```
del_timer(&timer1);
timer1.expires=jiffies+new_value;
add_timer(&timer1);
```

You can do this by using *mod_timer(&timer1, new_value)*; if the timer is expired by the time *mod_timer* is called, it acts like *add_timer* and adds the timer again.

Implementing timer as an LKM

LKM (Loaded Kernel Module) is supported by many UNIX-like operating systems and Microsoft Windows; it is an object file that is used for extending the kernel to support new hardware. Let's try to make a module that implements timer functions, to show you how to use timers in practice. For you to try this, you must have the kernel headers package installed. I have Fedora 17 installed, so I used *su -c 'yum install kernel-devel'* to install the specific headers. You may need to install *'kernel-PAE-devel'* if you are using the PAE kernel. Debian and Ubuntu users may use the *apt-get install module-assistant to* install the packages necessary to build an LKM.

After you have finished installing the packages, consider the following program:

```
#include<linux/kernel.h>
#include<linux/module.h>
```

Figure 1: Compiling the timer_example.c

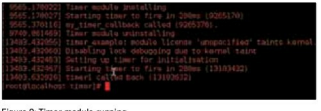

Figure 2: Timer module running

Figure 3: Timer module removed

```
#include<linux/timer.h>

/*create a timer named timer1*/
static struct timer_list timer1;

/*called when timer expires*/
void timer1_expires(unsigned long data)
{
 printk("timer1 called back (%ld)\n", jiffies);
}

int init_module(void)
{
 int retval;
 printk("Setting up timer for initialisation \n");
 setup_timer(&timer1, timer1_expires,0);

 printk("Starting timer to fire in 200ms (%ld)\n", jiffies);
 retval=mod_timer( &timer1, jiffies + msecs_to_jiffies(200) );
 if(retval) printk("Error in mod_timer\n");
 return 0;
}

void cleanup_module(void)
{
 int retval=del_timer(&timer1);;

 if(retval) printk("Timer still in use\n");
```